Discussion On Java

Discussion 01

IDE

Integrated **D**evelopment **E**nvironment (**IDE**): Software application that provides comprehensive facilities to computer programmers for software development.

We will use IntelliJ IDEA Community edition.

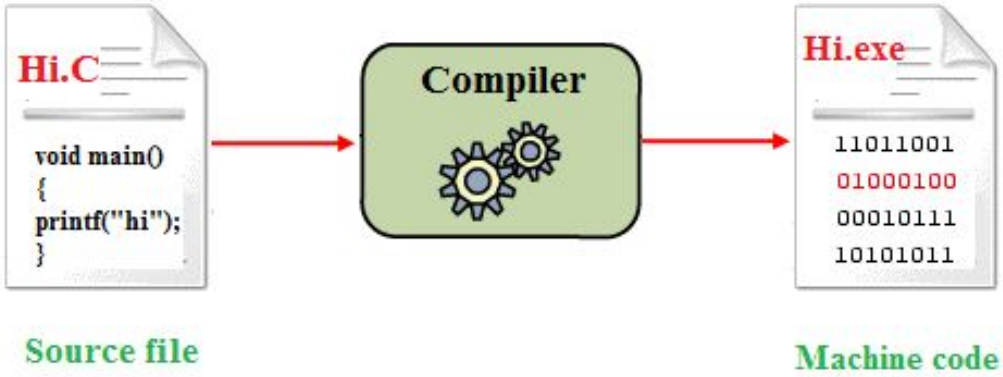
Structured Programming Language

- Programs are divided into small programs or functions.
- Less secured.
- Difficult to reuse code.
- Suitable for moderately complex task.
- E.g. C, Pascal etc.

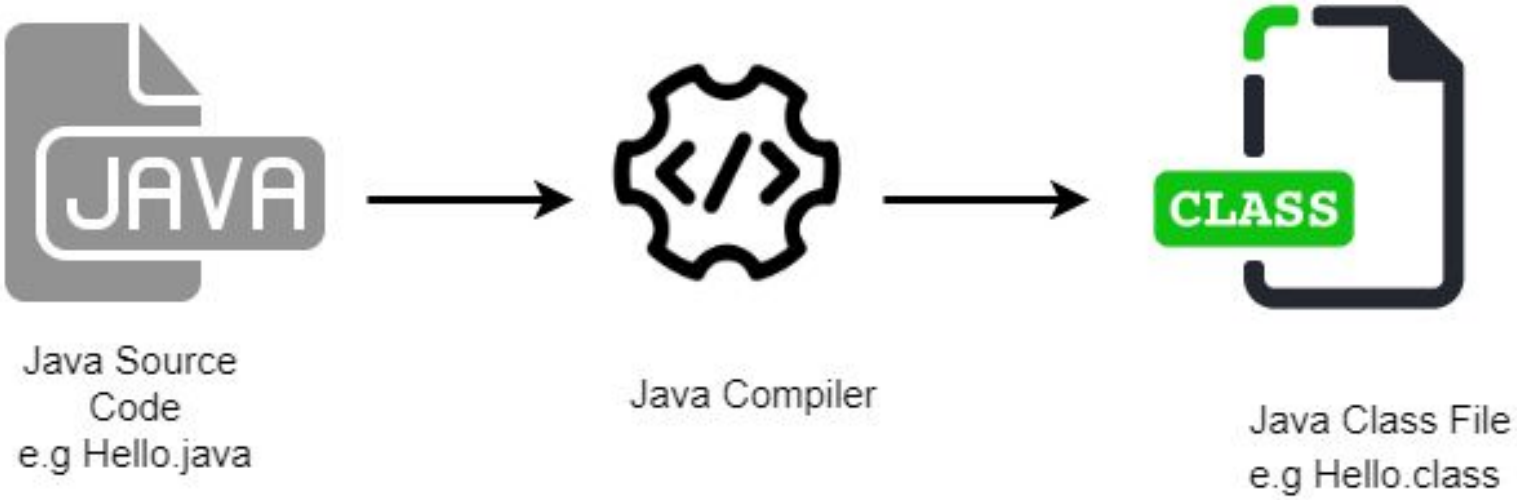
Object Oriented Programming (OOP) Language

- Works on data models or object.
- More secured.
- Good with reusable code.
- Can cope with large scale application.
- E.g. Java, Python, C# etc.

Compiler

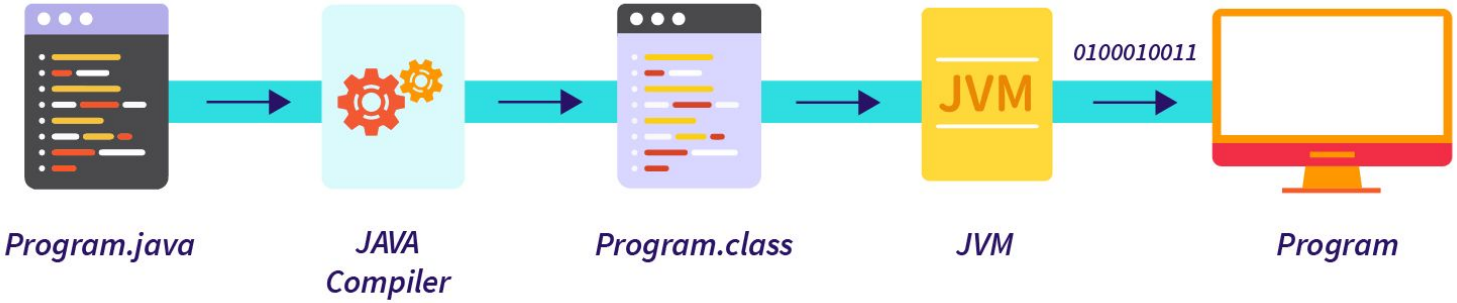

Hi.C
void main()
{
printf("hi");
}
Compiler
Hi.exe
11011001
01000100
00010111
10101011
Source file
Machine code

Java Compiler



.class file contains **BYTECODE**

How Does Java Programming Language Work?



SCALER Topics

Details: https://www.scaler.com/topics/java/how-java-program-works/

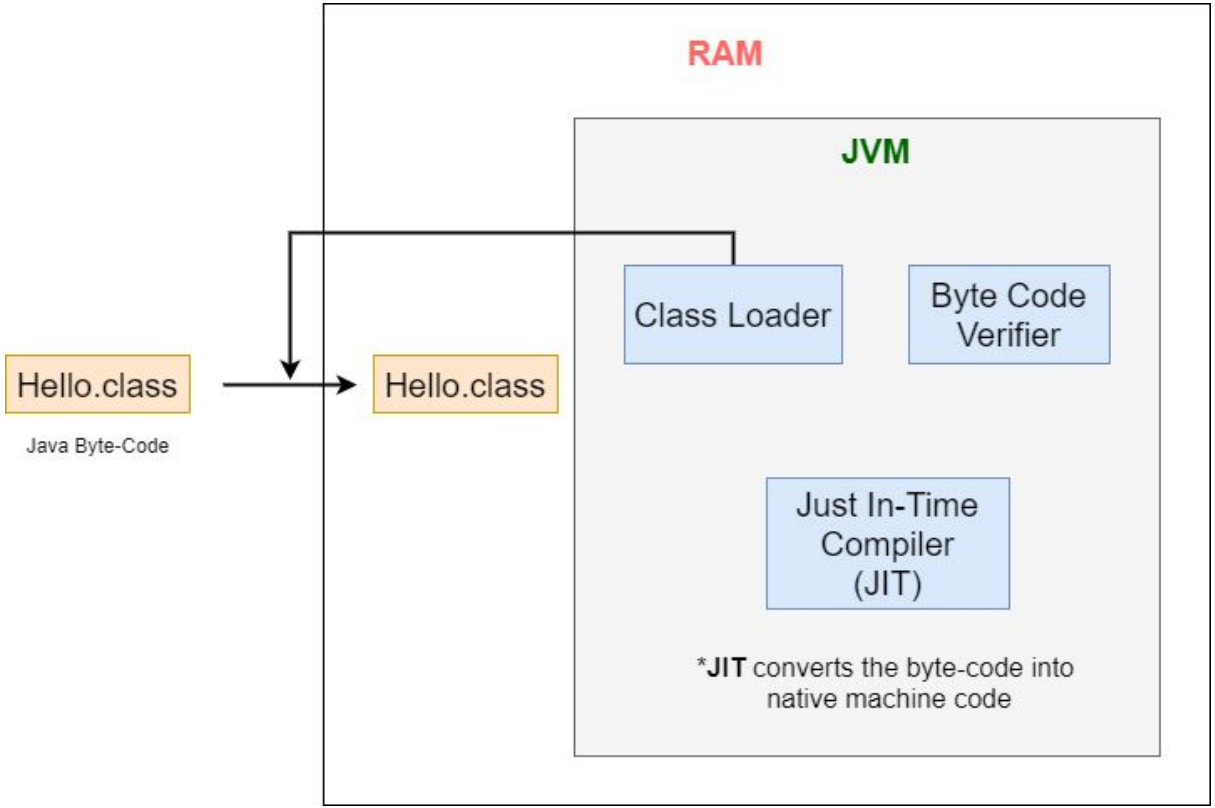
Some key terms comes with Java Architecture

- JVM: Java Virtual Machine.
- JRE: Java Runtime Environment.
- JDK: Java Development Kit

JVM

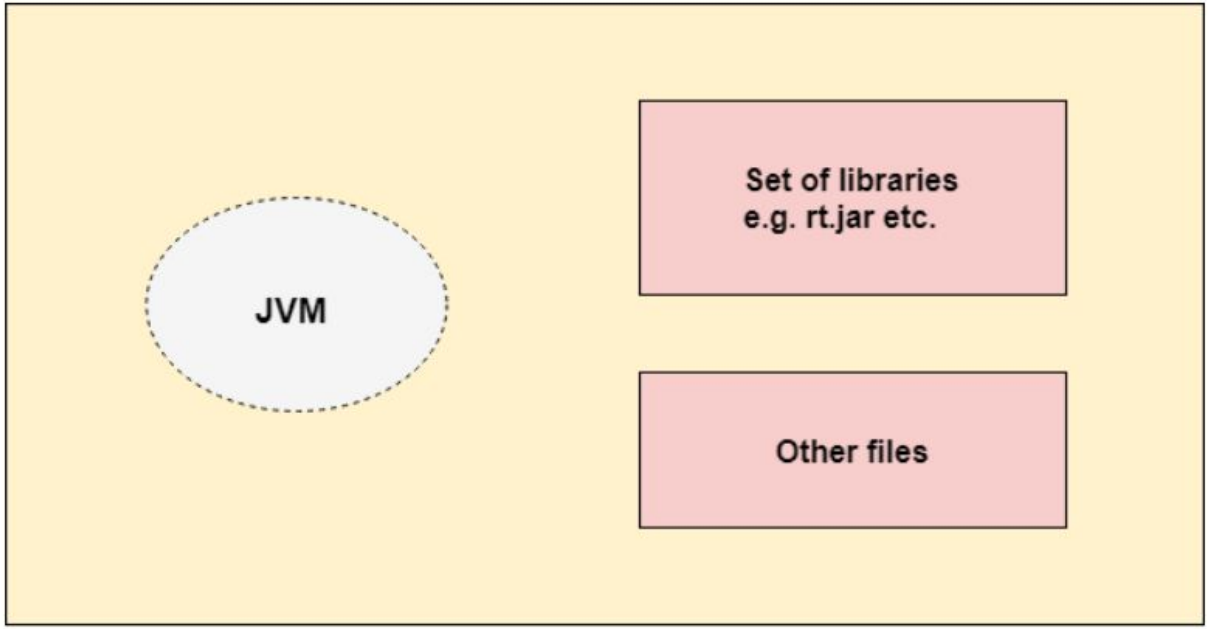
- Provide environment to process and execute Java Bytecode.
- Platform dependent means different OS needs different JVM.

JVM contd.


RAM
JVM
Class Loader
Byte Code Verifier
Hello.class
Java Byte-Code
Hello.class
Just In-Time Compiler (JIT)
*JIT converts the byte-code into native machine code

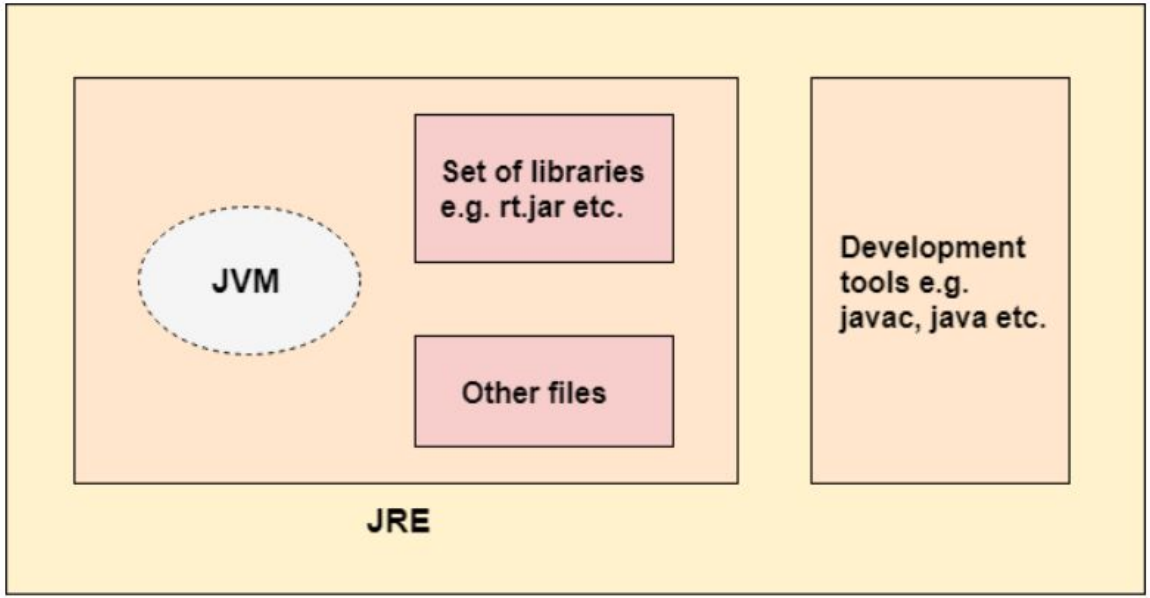
JRE

Contains JVM + other libraries.



JDK

Contains everything we need for development.



A First Simple Program

Identifier
: name of the class

Keyword
: to declare a class

All program activity occurs within me...

```
class introduction {

    public static void main(String args[]) {

        System.out.println("Hello World!");

    }

}
```

Class definition

Access Modifier
: control visibility of class member

methods body

Return type

method name
: the program will start executing from main()

(parameters)
: variables that catches any info passed to a method

A First Simple Program: Contd.

< introduction.java >

```
class introduction {

   public static void main(String[] args) {

        System.out.println("Hello World!");

   }

}
```

< introduction.java >

> **Javac introduction.java**

< introduction.class >

> **Java introduction**

Variables and Data Types

Basic Data Types are:

- Integers
- Floating-Point
- Character
- Boolean

Integer

Java defines 4 integer types:

- byte
- short
- int
- long

Integer contd.

Name	Width	Range
long	64	–9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
int	32	–2,147,483,648 to 2,147,483,647
short	16	–32,768 to 32,767
byte	8	–128 to 127

Floating-Point

- float
- double

Name	Width In Bits	Approximate Range
double	64	4.9e–324 to 1.8e+308
float	32	1.4e–045 to 3.4e+038

Character

- Keyword: char
- Needs 16 bit of storage.
- Represents character in *Unicode.*
- Can represents 65,536 symbols and character.

Boolean

- Keyword: boolean
- Only have two possible values: ***true*** or ***false***

Operators

- Arithmetic Operators
- Bitwise Operators
- Relational Operators
- Boolean Logical Operators etc.

Arithmetic Operators

Operator	Result
+	Addition
–	Subtraction (also unary minus)
*	Multiplication
/	Division
%	Modulus
++	Increment
+=	Addition assignment
–=	Subtraction assignment
*=	Multiplication assignment
/=	Division assignment
%=	Modulus assignment
– –	Decrement

Relational Operators

Operator	Result
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Boolean Logical Operators

Operator	Result
&	Logical AND
|	Logical OR
^	Logical XOR (exclusive OR)
||	Short-circuit OR
&&	Short-circuit AND
!	Logical unary NOT
&=	AND assignment
|=	OR assignment
^=	XOR assignment
==	Equal to
!=	Not equal to
?:	Ternary if-then-else

Conditional Statement

- If statement
- If - else if chain
- If - else if - else chain

Practice Problem 01

- Find the area of a triangle : *0.5 * base * height*
- Find the volume of a ball: *4/3 * pi * radius^3*
- Find the area of a circle: *pi * radius^2*
- Convert Celsius scale to Fahrenheit: *C / 5 = (F - 32) / 9*
- Find the area of a equilateral triangle: *(√3 / 4) * arm^2*
- Find the area of a triangle (2) : *√(s *(s-a)*(s-b)*(s-c))*

where *s = (a+b+c) / 3*

Practice Problem 01 contd.

- Write a program to check whether a given input is a positive integer or not.
- Write a program to check whether a given input is an odd or even number.

Hint: Use the remainder (%) and the equal (==) operators

Practice Problem 01 contd.

- Write a program to calculate grade of the following scenario:

  0 - 39 : F

  40 - 59 : C+

  60 - 69 : B

  70 - 79 : A-

  80 - 89 : A

  90+ : A+

Practice Problem 01 contd.

- Build a simple calculator.

  Input1: Integer a

  Input2: Integer b

  Input3: Any of the following symbols : + or, - or, * or, /

  Output: Print the result as the operation expressed by the Input3.

  Hint: use ***char*** while taking inputs and, **==** operator in *if-statements.*

```
Scanner scan = new Scanner(System.in);
Char ch = scan.next().charAt(0);
```

Discussion 02

Variable Naming Conventions of Java

- All identifiers should begin with a letter (A to Z or a to z), currency character ($) or an underscore (_).
- After the first character, identifiers can have any combination of characters.
- A keyword cannot be used as an identifier.
- Most importantly, identifiers are case sensitive.
- **Examples of legal identifiers**: *age, $salary, _value, __1_value.*
- **Examples of illegal identifiers**: *123abc, -salary.*

Nested If-statement

```
if (expression1){
// body statements - can be blank
    if (expression2){
    // body of the child conditional
    }
}
```

Increment and Decrement

- ++
- --
- +=
- -=
- *=
- /=
- Pre-increment: ++*var*
- Post-increment: *var*++

Loop Control Statement

- for Loop
- while Loop
- do-while loop

for Loop

```
for(initialization; condition; iteration) {
// body
}
```

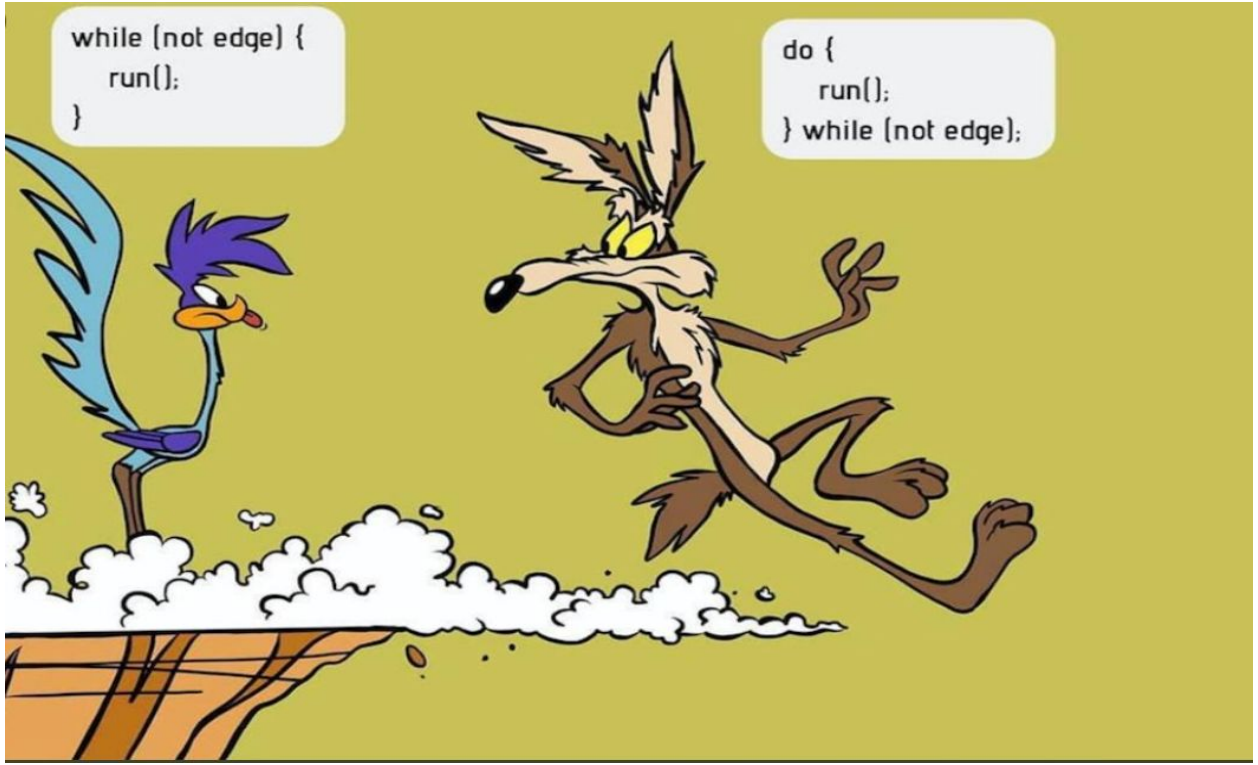
while Loop

```
initialization;
while(condition) {
// body of loop
iteration;
}
```

do-while Loop

```
initialization;
do {
// body of loop
iteration;
} while (condition);
```

while vs do-while loop



Jump Statement

- break
- continue

Practice Problem 02

- Calculate if a given year is a leap year or not.

  Hint: A leap year is divided by 400 or it is divided by 4 but not divided by 100.

- Print the following series (using for-loop, while loop & do-while loop as well):
  - 2, 4, 6, 8, 10, 12, 14, 16, 18, 20
  - 1, 3, 5, 7, 9, 11, 13, 15, 17, 19
- Calculate the sum of the series above (using the loops)

Discussion 03

Nested Loop

```
for(initialization1; condition1; iteration1) {
// body: Outer Loop
for(initialization2; condition2; iteration2) {
    // body : Inner Loop
    }
}
```

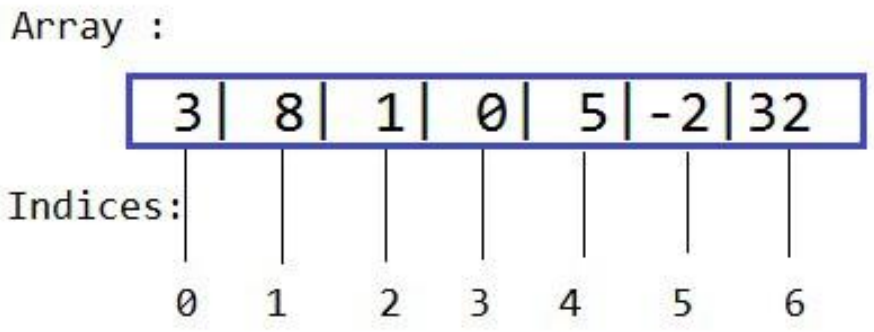
Nested Loop contd.

```
for(initialization1; condition1; iteration1) {
// body1
while(condition2) {
    // body2
    }
}
```

Jump Statements in Nested Loop

- In Inner Loop
- In Outer Loop

Arrays



Practice Problem 03

- Store some values (integer or floating point) and print the average of them.

  Hint: use Array and loops

- Write a program to give medel if a student has a letter grade and completed the semester. Use nested If statement.

  Hint: letter grade >= 3.5

- Consider the following scenario for giving bonus:
  - Reached 95% of target sales and had an attendance of 100% = 60% bonus
  - Reached 95% of target sales and had an attendance of 90% = 40% bonus
  - Reached 80% of target sales and had an attendance of 100% = 40% bonus
  - Reached 80% of target sales and had an attendance of 90% = 20% bonus
  - None of above = 5% bonus

Practice Problem 03 contd.

- Find the greatest number of a given series. Use Array and loops.
- Write a program to see if a given number is a prime number or not.

  Hint: A given number is a prime if and only if it is divided by 1 and the number itself.

- Reverse a given series. Use Arrays and loops.

Practice Problem 03 contd.

- Store some values and print how many odd and even number are in there.
- Print the following Fibonacci series:

  0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89

  Hint: The next number will be the summation of the two previous elements. E.g. 5 = 2 + 3, 89 = 34 + 55

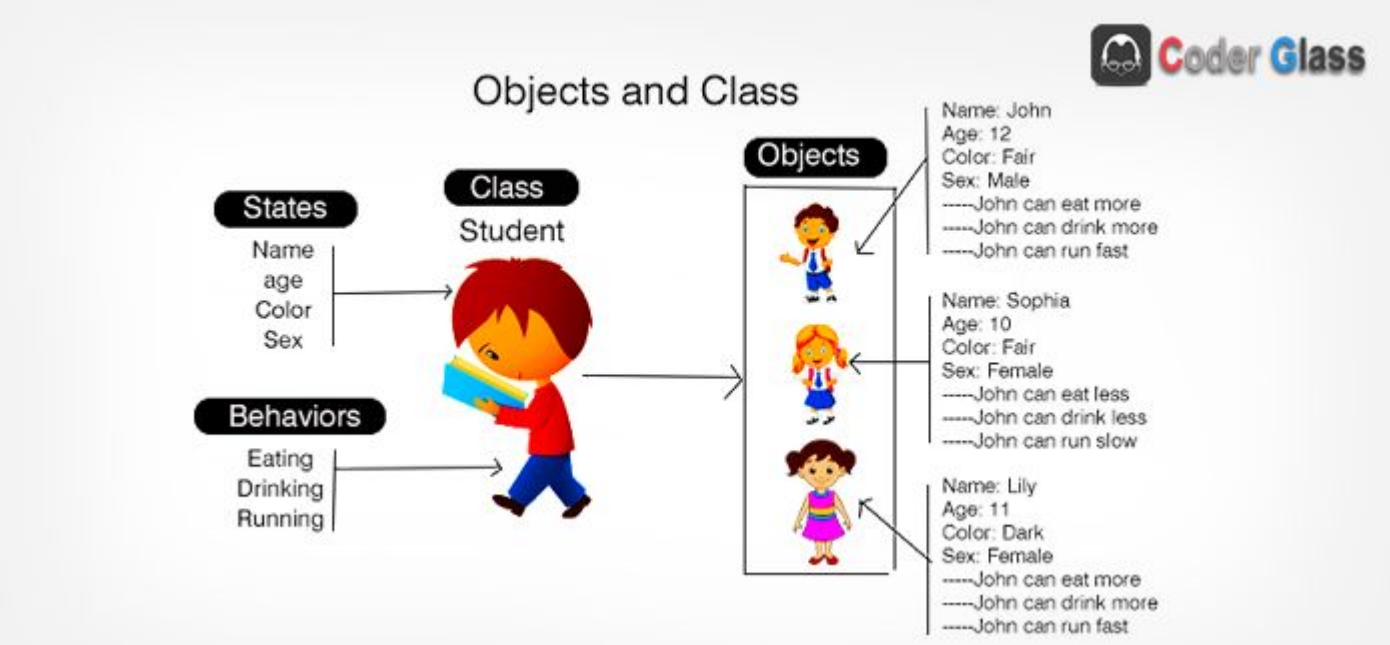
Discussion 04

2D Arrays

	Column 0	Column 1	Column 2
Row 0	x[0][0]	x[0][1]	x[0][2]
Row 1	x[1][0]	x[1][1]	x[1][2]
Row 2	x[2][0]	x[2][1]	x[2][2]

Object and Classes

A **Class** is a blueprint for **Objects**!



Methods

return_type method_name(*parameter_list*){

// body

}

Methods(Contd.)

```
int square(int num) {
  return num * num;
}
...
...
result = square(10);
// code
```

return value

method call

Methods(detailed declaration)

```
modifier static returnType methodName( parameter_list){
// body
 }
```

Modifiers

1. **Definition**: Specifies the **access level** or **behavior** of the method.
2. **Possible Values**:
   - public: Accessible from any class.
   - private: Accessible only within the class.
   - protected: Accessible within the package and subclasses.
   - *No Modifier (Default)*: Accessible within the same package.

static

1. **Definition**: Indicates that the *method* belongs to the class, not an instance of the class.
2. **Optional**: If omitted, the *method* belongs to the instance.
3. **Usage**:
   - static: Accessed without creating an object (ClassName.methodName()).
   - No *static*: Accessed through an object (objectName.methodName()).

Static: key characteristics

1. **Shared among all instances**
2. **Access without object**
3. **Memory efficiency**
4. **Can not access instance member(investigate)**

Static(Contd.)

1. **Static variable**
2. **Static methods**
3. **Static blocks**
4. **Static nested class**

Static(Exercise)

Imagine a library system where you need to track the total number of books issued, irrespective of the users.

Method Overloading

```
public class class_name{
type method_name(parameter_list_1){
 // method body
      }
type method_name(parameter_list_2){
 // method body
      }
      ...
}
```

Method Overloading(Contd.)

- By Changing the *number of params*

```
void sum(int a, int b){...}
void sum(int a, int b, int c){...}
```

- By Changing the *data type of params*

```
void sum(int a, int b){...}
void sum(string a, string b){...}
```

Method Overloading(Contd.)

- Can we just change the return_type of the method and call it method overloading?

The method **signature** in Java consists of:

1. **Method name**
2. **Number of parameters**
3. **Types of parameters.**
4. **Order of parameters.**

```
int sum(int a, int b){...}

void sum(int a, int b){...}
```

return_type is **not** part of the **method signature**, the compiler cannot differentiate between the two methods, leading to a **compile-time error**

As when we call a method, we don't specify the return type which leads to ambiguity

Advantages of Methods

Code Reusability:
Methods allow you to write code once and reuse it multiple times, reducing duplication.

Modularity:
Code is organized into smaller, manageable parts (methods), making it easier to read and debug.

Simplifies Code Maintenance:
Changes in logic need to be updated only in the method instead of multiple places.

Improved Readability:
Methods provide meaningful names for tasks, improving code readability and understanding.

Ease of Testing:
Methods can be tested independently, ensuring correctness without testing the entire program.

Good to read!

- Scripting Language vs. Programming Language
- Java vs. JavaScript
- Multithreading in Java vs JavaScript Eventloop (Should read it later)

Practice Problem 04

- Print the following pattern:

  1

  1 2

  1 2 3

  1 2 3 4

  1 2 3 4 5

  Hint: Use nested loop

Practice Problem 04(Contd.)

- Design a product filtering system for an e-commerce platform where users can search for products based on different criteria such as:
  - By **category** (e.g., "electronics", "fashion").
  - By **price range** (e.g., products between $100 and $500).
  - By **brand** (e.g., "Apple", "Nike").
  - By **category and price range**.
  - By **category, price range, and brand**.

Implement this functionality using method overloading, where the **filterProducts** method is defined multiple times with varying parameters to handle these use cases. Each overloaded method should print what the user is searching for.

Discussion 05

Access Control

- Public
- Private
- Protected
- Static
- Final

Constructor

```
public class class_name {
    public class_name(list_of_parameters) { // Constructor
    }
    // class body
}
```

Constructor(Contd.)

- No-Arg Constructor - a constructor that does not accept any arguments
- Parameterized constructor - a constructor that accepts arguments
- Default Constructor - a constructor that is automatically created by the Java compiler if it is not explicitly defined.

Constructor Overloading

```
public class class_name{
class_name(parameter_list_1){
 // constructor body
      }
class_name(parameter_list_2){
 // constructor body
      }
      ...
}
```

Constructor(Summary)

- Constructors are invoked implicitly when you *instantiate objects*.
- The two rules for creating a constructor are:
  1. The name of the constructor should be the same as the class.
  2. A Java constructor must not have a return type.
- If a class doesn't have a constructor, the Java compiler automatically creates a ***default constructor*** during run-time. The default constructor initializes instance variables with default values. For example, the int variable will be initialized to 0
- A constructor cannot be abstract or static or final.
- A constructor can be overloaded but can not be overridden.

Constructor(Default)

Data Type	Default Value (for fields)
byte	0
short	0
int	0
long	0L
float	0.0f
double	0.0d
char	'\u0000'
String (or any object)	null
boolean	false

Questions#01

What is a top-level class? Why can't we declare top-level classes with the private access modifier in Java?

Hint: Think about class visibility rules in a Java file.
Example: What happens if we try private class MyClass?

Questions#01

Access Modifiers Recap for Classes

Modifier	Top-Level Class Allowed?	Meaning for Classes
`public`	✅ Yes	Class is accessible from anywhere in the project.
Default	✅ Yes	Class is accessible only within the same package.
`protected`	❌ No	Intended for inheritance, not applicable to top-level.
`private`	❌ No	Restricts access only within the containing class (irrelevant for top-level).

Questions#02

Why can't we assign a new value to an instance variable outside a method or constructor in Java?

Hint: Think about where variable assignments are allowed and how the compiler identifies valid statements.
Example: Why does `lang = "Python";` outside a method cause an error?

Questions#03

What are initializer blocks in Java, and how do they differ from constructors?

Hint: Consider their order of execution and purpose.
Example: When is a static initializer block executed compared to an instance initializer block?

Questions#04

What is the order of execution for static initializer blocks, instance initializer blocks, and constructors in Java?

Hint: Create an example with all three to see the sequence. Example: Add a static, an instance block, and a constructor to a single class and test it.

Discussion 06

String

- We can create a string with literal and new keyword
- String type: Primitive or Object?
- Differences between string creation with literal and new keyword

String(Contd)

- Array of Character
- Some useful methods:
  - length()
  - charAt()
  - equals()
  - substring()
  - concat()
  - toLowerCase()
  - toUpperCase()
  - trim()
  - indexOf()
  - replace()
- “equals()” vs “==”

String Creation(String literal vs new operator)

- https://dev.to/bad-logic/java-strings-understanding-string-literal-and-new-operator-3b4m

Practice problem 04-String

- Take a string as input and output reverse
- Check if a string is a palindrome or not
- Count the occurrence of a character

Discussion 07

this keyword

- ***this*** keyword is used to refer to the current object inside a method or a constructor
- Differentiate between instance variables and parameters(ambiguous variable naming)
- Call other constructors within the same class(used in constructor overloading, constructor chaining)
- Returning the Current Class Instance(enables method chaining)
- Passing this as an Argument

Practice Problem 05

- Consider the following scenario:

  A car has these following attributes. It has an owner, brand name, serial number etc. It also has these following functionalities. You can start/stop a car, its fuel can be checked etc.

  Now design a Class for this scenario.

Practice Problem 05 contd.

- Consider the following scenario:

  Suppose you have pet dog. It has a name, age, breed etc. It has the following characteristics. It can bark, do spin, run etc.

  Now create a class for this scenario.

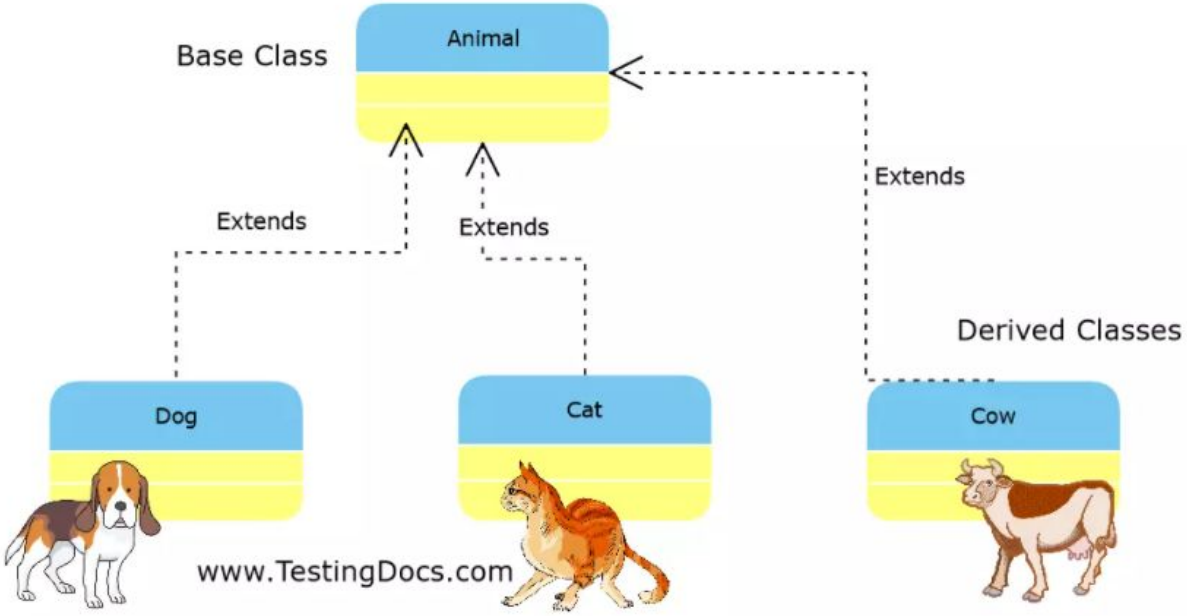
Practice Problem 05(contd.)

- Consider a library system where books have attributes like title and author. We can create a book with title or both.

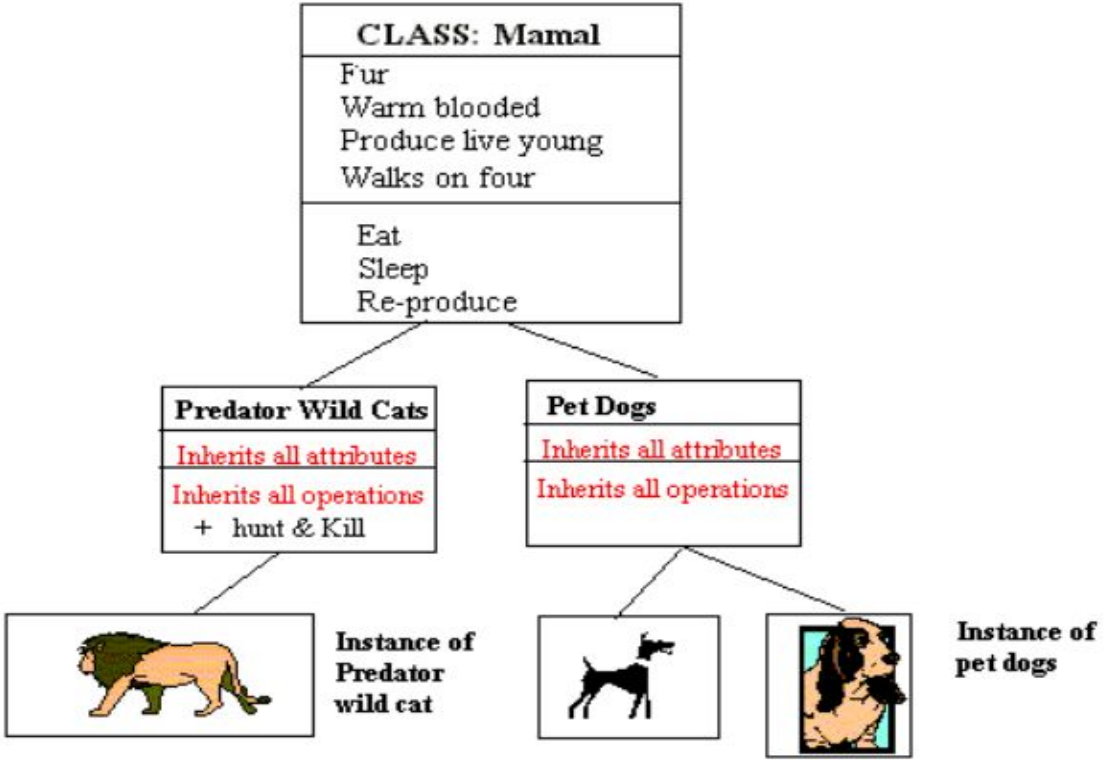
  How can we do that with constructor overloading and setting only the instance variable passed while object creation

Discussion 08

Inheritance (Extending Classes)



Inheritance (Extending Classes) contd.



Inheritance Syntax (Extending Classes)

```
public class A{
 // body of class A
}
public class B extends A{
 // body of B
}
```

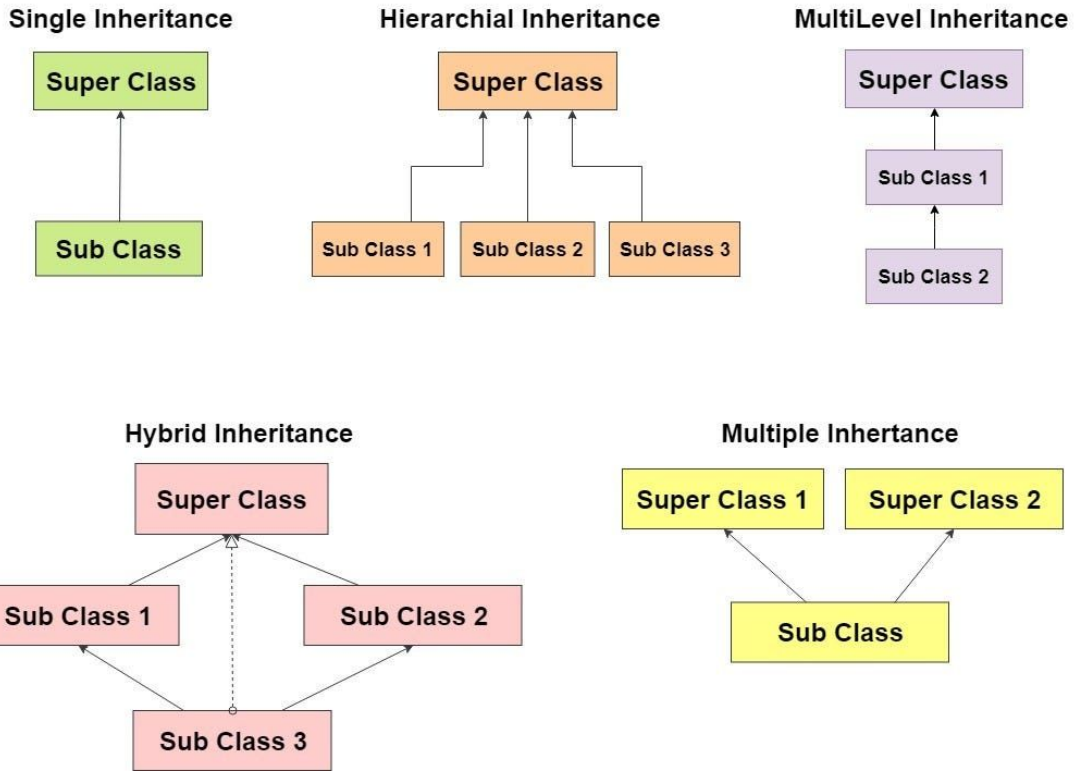
Inheritance

- Inheritance is a mechanism where a class (**child class**) **inherits** properties and behaviors from another class (**parent class**).
- The parent class is called the **superclass**, and the child class is called the **subclass**.
- In Java, inheritance is implemented using the `extends` keyword.
- Inheritance is an **is-a** relationship. We use inheritance only if there exists an is-a relationship between two classes. E.g. *Car* is a *Vehicle; Orange* is a *Fruit*; *Dog* is an *Animal*

WHY?

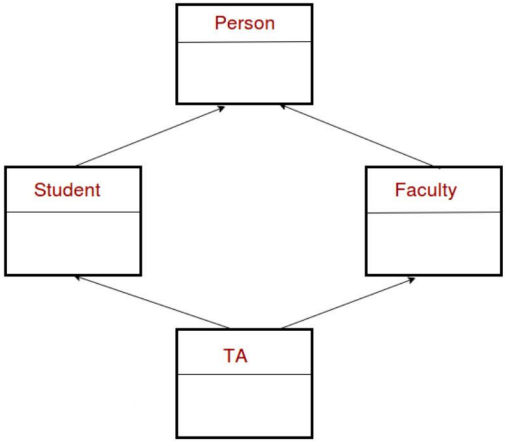
- **Code Reusability:** Avoids writing the same code multiple times.
- **Extensibility:** Allows modification and enhancement of existing code.
- **Hierarchy Representation:** Models real-world relationships (e.g., **Animal** → **Dog, Cat**).
- **Polymorphism:** Enables method overriding, leading to more dynamic code. Method overriding is also known as runtime polymorphism.

Inheritance Types



Diamond Problem

- The diamond problem can occur when two classes have a common parent as well as a common child (in case of multiple inheritance).



Discussion 09

Method Overriding

```
public class A{
 void m(){
        }
}
public class B extends A{
// overriding methods
 void m(){
        }
}
```

Method Overriding

What is Method Overriding?

- It allows a subclass to provide a specific implementation of a method already defined in its superclass.
- Used in **inheritance** to modify behavior without changing the parent class.

Why is it needed?

- To customize behavior in derived classes while keeping a common interface.
- Ensures flexibility and reusability in OOP.

Method Overriding

What Happens If We Don’t Use Method Overriding?

- If we do not override methods in subclasses, we would need **conditional logic in the superclass** to handle every possible subclass scenario.
- This **violates the Open-Closed Principle**

  (OOP rule: classes should be open for extension but closed for modification).

Rules of Method Overriding

1. **Same Method Signature** (name, return type, and parameters must be the same).
2. **Only in Subclasses** (child class must inherit from the parent class).
3. **Method Visibility Should Not Be Reduced**
   a. `public` in the parent → must remain `public` or `protected`.
   b. Cannot override a **private** method.
4. **Final Methods Cannot Be Overridden**
5. **Static Methods Are Not Overridden** (they are hidden instead).

Method Overriding X static, final

Feature	Static Method	Final Method
Can be inherited?	❌No (method hiding)	Yes
Can be overridden?	❌ No (only redefined in child class)	❌ No
When resolved?	Compile-time	Runtime
Use case	Utility methods, class-wide behavior	Prevent modification of core logic

Use of super keyword

The super keyword in Java is used in subclasses to access superclass members (attributes, constructors and methods)

1. To call **methods** of the superclass that is **overridden** in the subclass.
2. To access **attributes** (fields) of the **superclass** if both superclass and subclass have attributes with the same name
3. To explicitly call superclass no-arg (default) or parameterized **constructor** from the subclass constructor

Can we override field/attributes?

✅ Methods can be overridden, but fields cannot.

✅ Fields are inherited but resolved at compile-time, not runtime.

✅ If a field exists in both parent and child, it hides (not overrides) the parent's field.

✅ To initialize an inherited field, use `super()`, not direct assignment.

```
class Parent {
    String name = "Restaurant";
}
class Child extends Parent {
    String name = "Fast Food"; // Hides parent's field, does NOT override it
}
```

Polymorphism

Polymorphism is an important concept of object-oriented programming. It simply means more than one form.

That is, the same entity (method or operator or object) can perform different operations in different scenarios.

We can achieve polymorphism in Java using the following ways:

1. Method Overriding (Runtime Polymorphism)
2. Method Overloading (Compile-time Polymorphism)
3. Operator Overloading

Polymorphism

Dynamic Method Dispatch

- Dynamic Method Dispatch in Java is the process by which a call to an overridden method is resolved at runtime (during the code execution).
- The concept of method overriding is the way to attain runtime polymorphism in Java.
- During the code execution, JVM decides which implementation of the same method should be called.
- The reference variable belongs to the superclass or parent class while the object is made up of the subclass or child class. It is also known as ***upcasting***.
- https://www.scaler.com/topics/runtime-polymorphism-in-java/

Static vs Dynamic Binding...

Dynamic Method Dispatch

```
public class A{
 void m(){
      }
}
public class B extends A{
 void m(){
      }
}
```

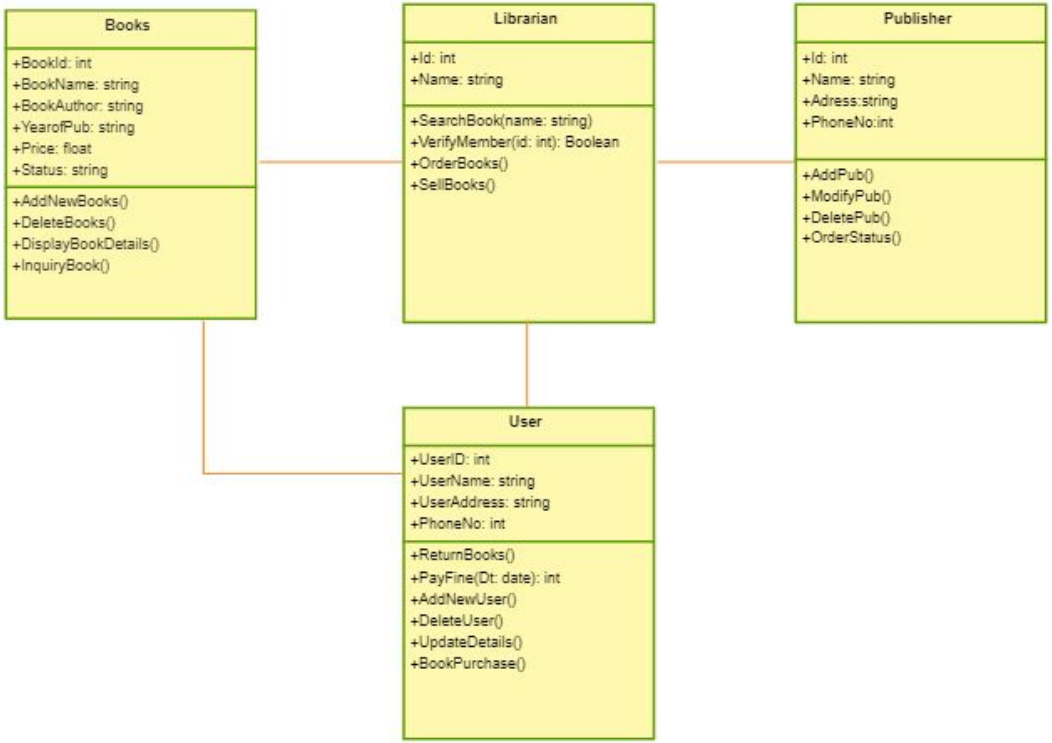
```
public class C extends A{
 void m(){
      }
}
A a;
B b = new B();
C c = new C();
a = b; a.m();
a = c; a.m();
```

Abstract Class and Method

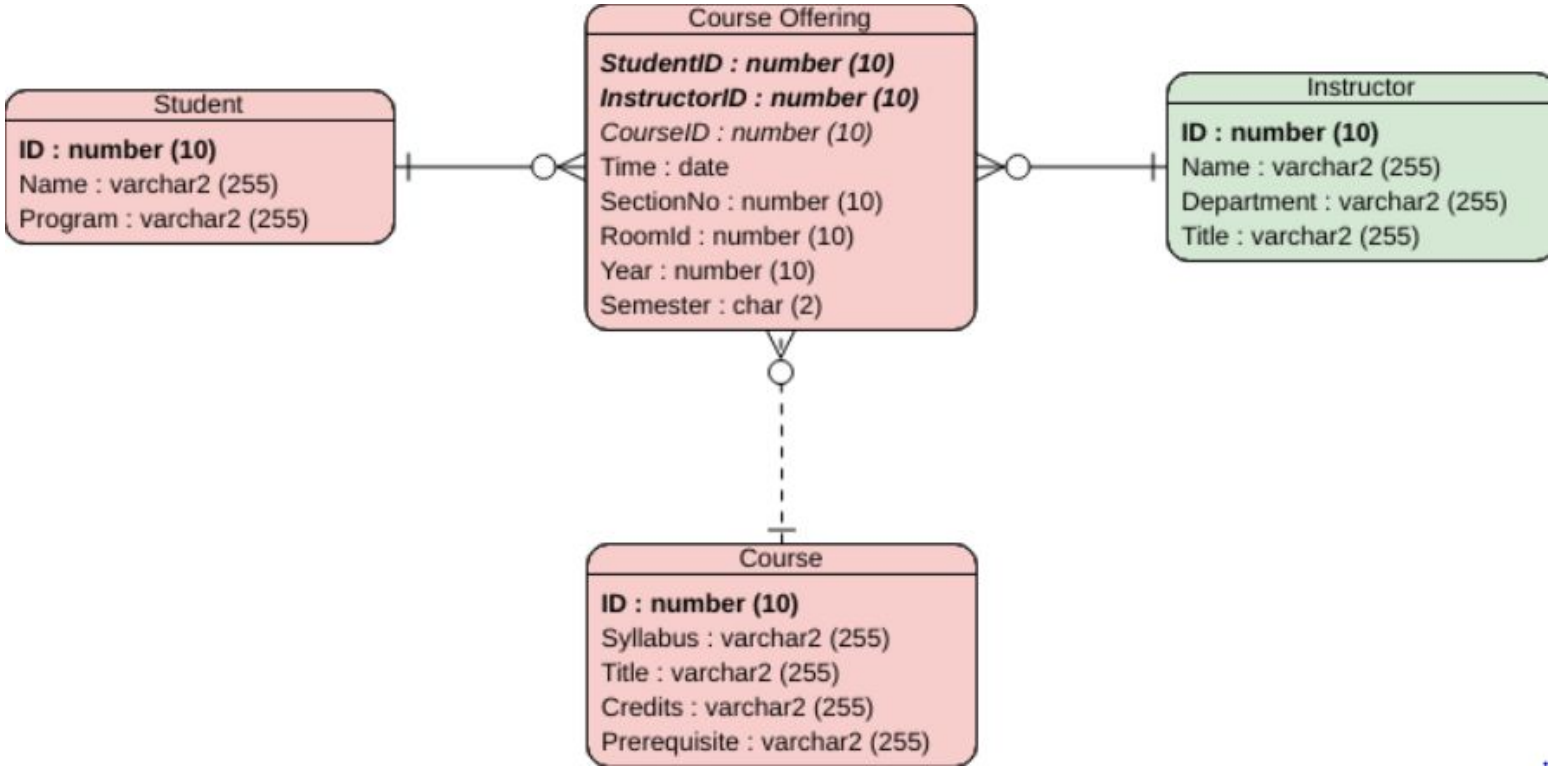
```
abstract class A{
abstract void m();
void n(){
        }
}
public class B extends A{
// overriding methods
 void m(){
        }
}
```

Discussion 10

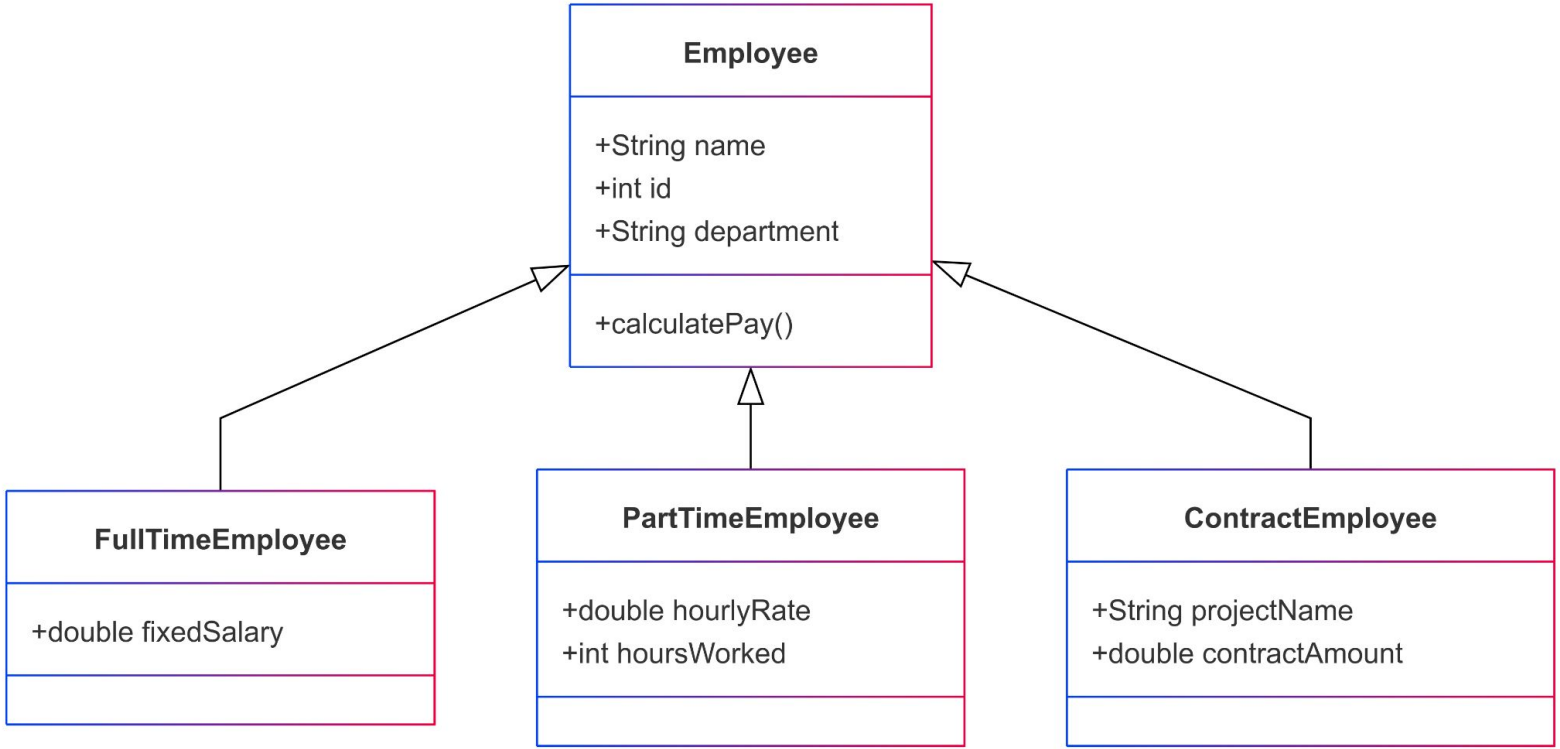
Practice Problem 06 : Implement the Scenario



Practice Problem 07 : Implement the Scenario



Practice Problem 08(a) : Implement the Scenario



Practice Problem 08(b) :

Imagine an **Online Food Ordering System** where different types of restaurants have different delivery times.

- A general **Restaurant** class provides a standard method for calculateTotalBill(), estimateDeliveryTime(40 mins).
- However, **Fast-Food restaurants** may have quicker delivery(20 mins), while **Fine-Dining restaurants** may take longer due to food preparation time(60 mins).
- A **Restaurant** has a standard **10%** tax. However, **FastFoodRestaurant** has a special fast food tax **of 15%** due to different regulations. The system should calculate the calculateTotalBill**(foodPrice + tax)** for both restaurant types.

Practice Problem 08(c)

- Consider the following scenario:

  Suppose there are five students in a classroom. They had to study five subjects for their final exam. Each subjects has a certain amount of credits or weights. Now calculate the CGPA for each student. (use 2D array)

  Hint: CGPA = sum_(Grade Point * Credits) / sum_(Credits)

Discussion 11

Interface

```
interface Interface_identifier{
    void method1();
    void method2();
    .
    .
    .
    }
```

```
interface Polygon {
  void getArea(int l, int b);
}

class Rectangle implements Polygon {
  void getArea(int l, int b) {
     System.out.println("Area:” + l*b);
   }
}
```

Multiple Interface(Multiple Inheritance)

```
interface A {
  // members of A
}
interface B {
  // members of B
}
class C implements A, B {
  // abstract members of A
  // abstract members of B
}
```

Extending Interface

```
interface A {
  // methods of interface A
}

interface C extends A {
   // methods of interface C
   // methods of interface A
}
```

```
interface A {
  ...
}
interface B {
  ...
}
interface C extends A, B {
  ...
}
```

Extending Interface(multiple)

Would there be any problem like Diamond problem?

```
interface A {
   ...
}
interface B {
   ...
}
interface C extends A, B {
   ...
}
```

Interface

An **interface** in Java is a **blueprint** of a class that defines a set of methods without implementation.

- All methods in an interface are *abstract* (by default, public and abstract)
- An interface cannot have *instance variables*, but it can have *constants* (public static final)
- We can not create *instances* of an interface
- A class implements an interface using the *implements* keyword

Key Features:

✔ Provides full abstraction (only method signatures, no implementation).
✔ Supports multiple inheritance in Java (unlike classes).
✔ Ensures a contract that classes can follow (specification class must follow)

Interface

All the *methods* inside an interface are implicitly public and all *fields* are implicitly public static final

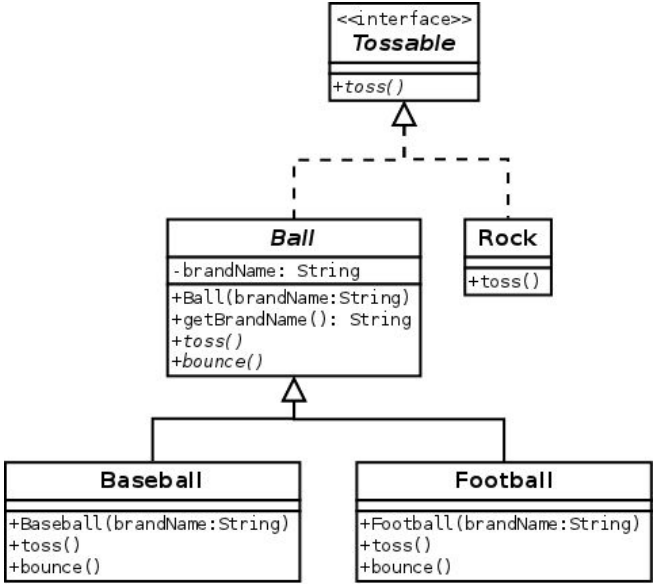
```
interface Language {
  //
  String type = "programming language";


  //
  void getName();
}
```

Interface (Default methods)

With the release of **Java 8**, we can now add <u>methods with implementation</u> inside an interface. These methods are called ***default methods.***

```
interface Language {
  // default methods
  public default void getVersion() {
        System.out.println("Current version is 1.0");
  }
}
```

Interface (Class diagram)



https://w3.cs.jmu.edu/spragunr/CS239/lectures/interfaces/interface_worksheet_solutions.shtml

Encapsulation & Data hiding

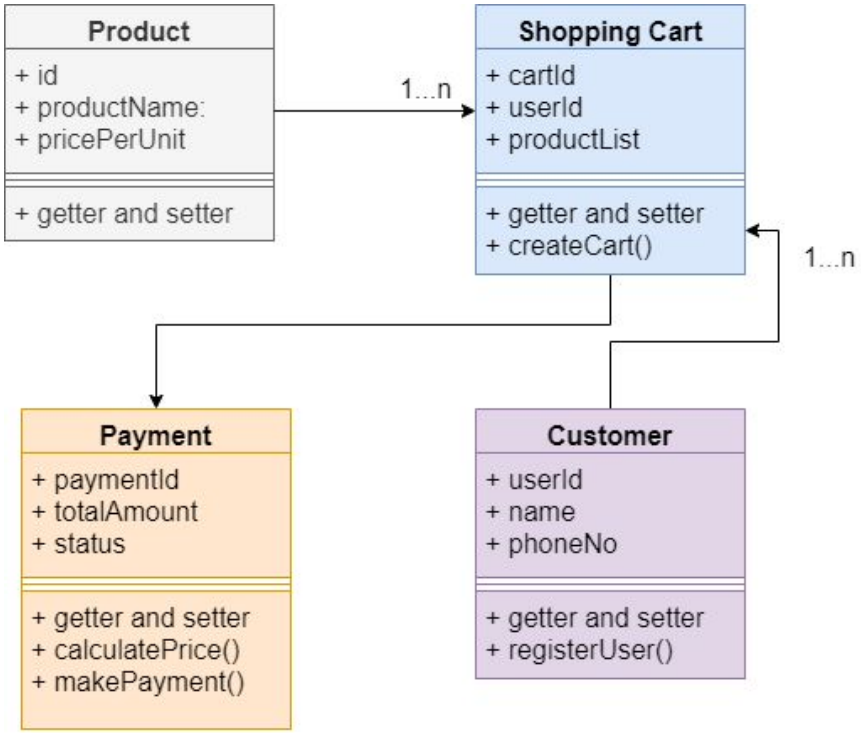
Encapsulation is one of the key features of object-oriented programming. Encapsulation refers to the bundling of fields and methods inside a single class.

Data hiding is a way of restricting the access of our data members by hiding the implementation details. Encapsulation also provides a way for data hiding.

We can use access modifiers to achieve data hiding.

- https://stackoverflow.com/questions/12013448/encapsulation-vs-data-hiding-java
- https://stackoverflow.com/questions/28612420/what-is-encapsulation-exactly/28614407#28614407

Design The Following Scenario



Discussion 11

UML class diagram

The UML Class diagram is a graphical notation used to construct and visualize object oriented systems. A class diagram in UML is a type of static structure diagram that describes the structure of a system by showing:

- classes,
- their attributes,
- operations (or methods),
- and the relationships among objects.

- https://medium.com/@bindubc/association-aggregation-and-composition-in-oops-8d260854a446
- https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-class-diagram-tutorial/
- https://www.youtube.com/watch?v=6XrL5jXmTwM

Exception Handling

Keywords:

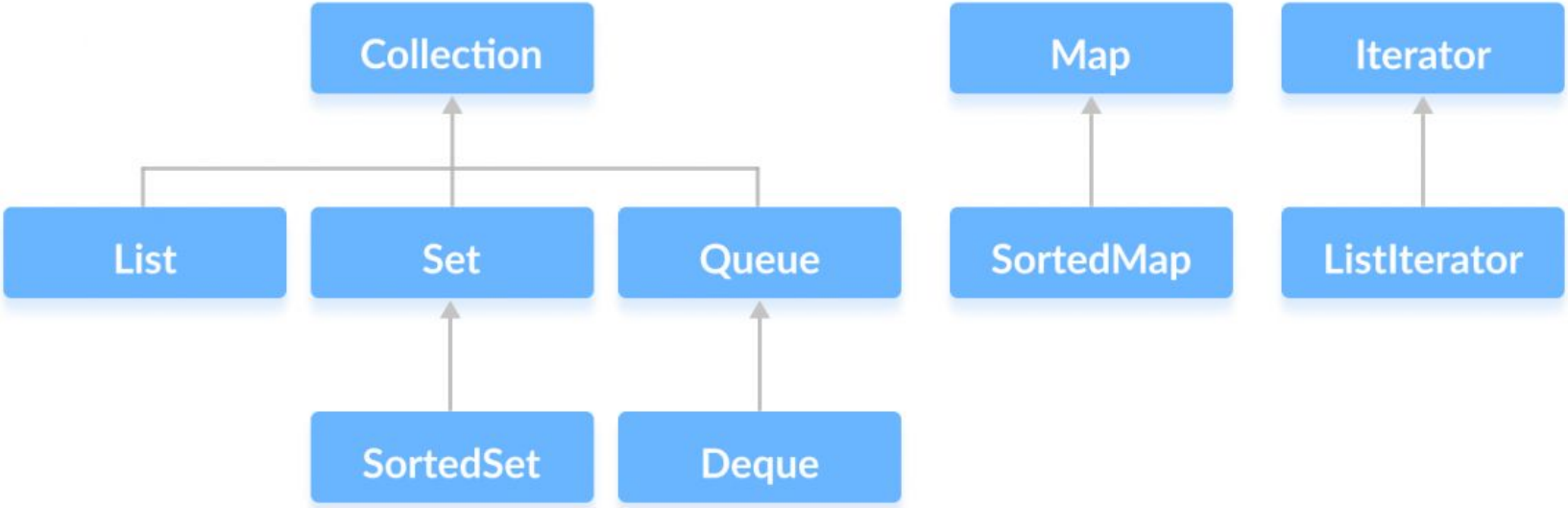
- try
- catch
- finally
- throw
- throws

Try-Catch statement

```
try{
    // fault prone code
  } catch(Exception e){
    // ……
  }
```

Discussion 12

Java Collections Framework



Some Common Collection Classes

- ArrayList
- HashSet
- Map
- Arrays
- Collections etc

Discussion 13

Some common terms comes with the Collections

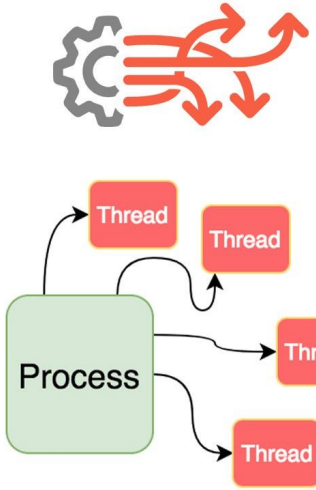
- Java Generics (term / feature)
- Iterator (class)
- Comparator (interface)

Discussion 14

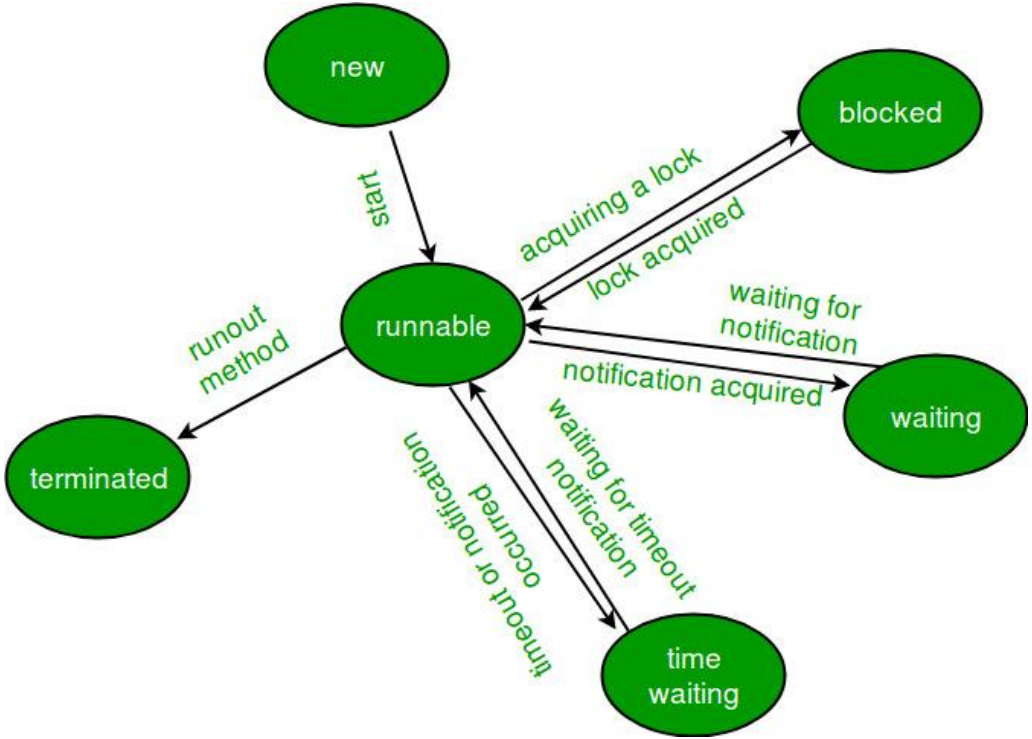
Thread

Two ways to implement thread:

- Extending **Thread** Class
- Implementing **Runnable** Interface.



Thread : Life-Cycle



Thread : Synchronized method

Usage :

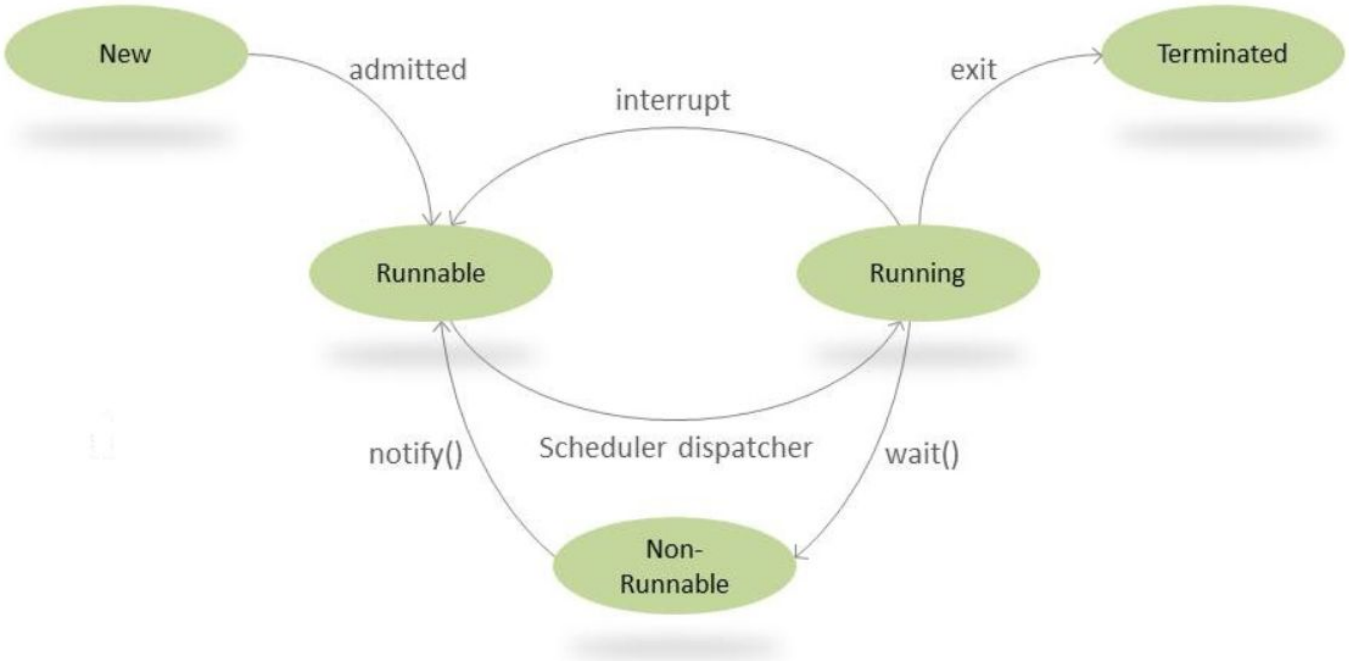
- To prevent thread interference
- To prevent consistency problem

Syntax:

```
synchronized method_signature method_name(){
    ....
    }
```

Discussion 15

Thread : Shifting between Runnable and Non-Runnable State



Some common terminology comes with Thread

- sleep()
- join()
- mutex lock
- wait()
- notify(), notifyAll()
- volatile

Interview Preparation

Preparation for Job and Central Viva (Pub)